

Adjoint Sensitivities and Gradient-Based Manning Roughness Calibration in RDycore

DRAFT for internal review

Mark Adams¹, [TODO: co-authors: Gautam Bisht, others TBD]

¹Lawrence Berkeley National Laboratory

Draft of August 18, 2026

Abstract

We present a discrete-adjoint capability for RDycore, the exascale river dynamical core of the Energy Exascale Earth System Model (E3SM), and use it for gradient-based calibration of spatially distributed Manning roughness. The capability comprises: an exact, assembled Jacobian of the finite-volume shallow-water right-hand side — closed-form Roe-flux blocks obtained by hand-written forward-mode differentiation of the flux computation, including the entropy-fix branches, together with source-term and boundary-condition blocks; adjoint sensitivities of gauge-misfit functionals with respect to both the initial state and the Manning field via PETSc’s `TSAdjoint`, at a cost of roughly one forward solve per gradient independent of the parameter count; and bound-constrained calibration with TAO. Every derivative is validated against finite differences of the full nonlinear solve at tight, pre-registered tolerances (Jacobian blocks to 10^{-8} , gradients to 10^{-5}). Twin experiments recover a two-zone roughness field to 4×10^{-7} relative error, and a per-cell field to 8% with multi-time observations and Tikhonov regularization. The same assembled Jacobian enables fully implicit backward-Euler stepping — which we show is required in practice: the explicit treatment of Manning drag is unstable at the operational time step of a Hurricane Harvey test case, in quantitative agreement with a friction-stiffness stability analysis — and the implicit integrator’s adjoint passes the same finite-difference gates. On a Hurricane Harvey twin configuration (2,746 cells, implicit stepping) the calibrated Manning map recovers a two-zone truth field to 19.4% with proper Tikhonov regularization, and the gauge-sparse setting exhibits — and lets us dissect — the classic semiconvergence of underdetermined friction estimation; a twin using the actual 17-gauge USGS network geometry fits its 238 observations essentially perfectly while recovering the field to only 45%, quantifying why real-data calibration must shrink the parameter space to match the network. To our knowledge this makes RDycore the first E3SM component model with built-in discrete-adjoint parameter estimation, without rewriting the solver in a machine-learning framework.

1 Introduction

Manning roughness is the dominant tunable parameter of depth-averaged flood models, and its calibration is traditionally manual or ensemble based. Adjoint methods deliver the gradient of an observation misfit with respect to *every* cell’s roughness at the cost of about one extra forward solve, making distributed calibration tractable; the approach is well established in shallow-water modeling, from channel networks [1] and unstructured finite-volume twin experiments [2] to GPU-accelerated optimizers [3], and is deployed in variational-assimilation platforms such as DassFlow [4, 5] and the Thetis coastal model, which calibrated a spatially varying Manning field for the Baltic Sea

with an automatically generated adjoint [6] after storm-surge sensitivity studies that addressed the overfitting inherent in distributed friction estimation [7].

A recent generation of *differentiable* flood and hydrology codes obtains gradients by rewriting simplified solvers inside machine-learning frameworks: Inunda (PyTorch, local-inertial, calibrated on a Hurricane Harvey hindcast) [8], Hydrograd (Julia) [9], AegirJAX [10], CaMa-Flood-GPU [11], and differentiable routing [12]. These demonstrate demand for exactly this capability, but none is a production Earth-system component: they trade the validated numerics, unstructured meshes, and coupling infrastructure of codes like RDycore for differentiability.

This paper takes the complementary route: make the production code differentiable. RDycore is a finite-volume shallow-water dynamical core (Roe fluxes, well balancing, wetting and drying) validated at up to 471M cells. We add the missing derivative infrastructure — an exact assembled Jacobian and TSAdjoint-based sensitivities [13] — entirely behind a configuration flag, with no impact on existing runs and no new dependencies.

Contributions: (i) an exact, FD-verified Jacobian of the SWE right-hand side, including Roe dissipation, entropy-fix branches, wet/dry branches, and reflecting/Dirichlet/critical-outflow boundary conditions; (ii) adjoint gradients dJ/du_0 and dJ/dn with respect to water-height observations at gauge locations at multiple observation times, FD-validated to 10^{-5} or better; (iii) TAO-based recovery of per-region and per-cell roughness fields in twin experiments, with a quantified observability study; (iv) implicit (backward Euler) stepping and its adjoint, enabled by the same Jacobian, with an empirical demonstration that explicit friction fails at operational time steps on a Hurricane Harvey configuration; and (v) a catalog of the nondifferentiable structures encountered — useful, we believe, to anyone retrofitting adjoints onto an operational flood code.

2 Forward model

RDycore solves the 2D shallow water equations in conservative variables $u = (h, q_x, q_y)$, $\mathbf{q} = h\mathbf{v}$:

$$\partial_t h + \nabla \cdot \mathbf{q} = Q_r(x, t), \quad (1)$$

$$\partial_t \mathbf{q} + \nabla \cdot \left(\frac{\mathbf{q} \otimes \mathbf{q}}{h} + \frac{gh^2}{2} \mathbb{I} \right) = -gh \nabla z_b - g n^2 h^{-7/3} \mathbf{q} \|\mathbf{q}\|, \quad (2)$$

with cell-centered finite volumes on an unstructured mesh:

$$\frac{du_i}{dt} = -\frac{1}{|\Omega_i|} \sum_{e \in \partial\Omega_i} \mathcal{F}(u_i, u_j, \mathbf{n}_e) L_e + S(u_i; n_i, t) \equiv f_i(u, n), \quad (3)$$

where $\mathcal{F}$ is the Roe flux with a critical-flow (entropy) fix, primitives are reconstructed with a regularized division and a dry cutoff $h_{\min}$, and S contains bed slope, Manning drag, and external forcing.

A differentiable right-hand side. A prerequisite that deserves emphasis: RDycore’s existing friction treatments (a semi-implicit splitting and the implicit scheme of Xia and Liang) embed the time step — and, in the semi-implicit case, the flux divergence — *inside* the RHS callback. The resulting map is consistent only with forward Euler and admits no well-defined $\partial f / \partial u$. We therefore added a plain explicit source treatment (`source.method: explicit`), used on every adjoint path; Section 6 shows how the resulting stiffness is then handled properly, by the integrator rather than inside f .

Check	Configuration	Result	Gate
Edge flux blocks	wet jump 10 m/5 m	7.6×10^{-9}	10^{-6}
Edge flux blocks	transcritical (entropy-fix branch)	2.4×10^{-10}	10^{-5}
Source block	shallow flowing state	$< 10^{-6}$	10^{-6}
Assembled matrix	uniform flow, reflecting BCs	1.6×10^{-8}	10^{-6}
Assembled matrix	dam break, reflecting BCs	1.5×10^{-8}	10^{-6}
Assembled matrix	dam break, Dirichlet + critical outflow	1.9×10^{-8}	10^{-6}
dJ/du_0 (explicit RK)	8 sampled components	1.0×10^{-8}	10^{-5}
dJ/dn (explicit RK)	domain aggregate	2.9×10^{-6}	10^{-5}
dJ/du_0 (implicit BE)	tight inner tolerances	1.4×10^{-8}	10^{-5}
dJ/dn (implicit BE)	tight inner tolerances	5.3×10^{-6}	10^{-5}

Table 1: Relative errors of analytic derivatives against central finite differences of the full nonlinear computation. All checks run in CI.

3 Exact Jacobian

The RHS Jacobian is block sparse with 3×3 blocks on the cell-adjacency graph: each interior edge (i, j) contributes $\mp(L_e/|\Omega_{i,j}|) \partial \mathcal{F} / \partial u_{L,R}$ to the two cell rows, local sources contribute diagonal blocks, and boundary edges contribute diagonal blocks through the ghost-state map of the boundary condition.

Flux blocks by forward-mode differentiation. Rather than deriving the dissipation-matrix derivatives symbolically, we differentiate the flux *computation*: a hand-written forward-mode (tangent) version of the Roe flux routine propagates directionals through every intermediate — Roe averages, eigenvectors, eigenvalues including both branches of the critical-flow fix, wave strengths, and physical fluxes — taking the code’s own branch at every $|\cdot|$ and switch (a consistent one-sided derivative). Seeding unit directions through the primitive-reconstruction chain rule yields the two conservative-space blocks in six tangent evaluations per edge. The approach mirrors the flux code line by line, which makes it reviewable and, in our experience, correct on first assembly against the verification harness below.

Boundary blocks. For a boundary edge, $F(u_{\text{in}}, u_g(u_{\text{in}}))$ contributes $(\partial_L F + \partial_R F \partial u_g / \partial u_{\text{in}})$. The reflecting map is a linear mirror; Dirichlet ghosts are constant; critical outflow has a closed-form ghost map $h_g = (q^2/g)^{1/3}$ with the code’s inflow branch producing an identically zero flux.

Parameter Jacobian. Manning n appears only in the drag term, so $\partial S_{\mathbf{q}} / \partial n = -2gnh^{-7/3} \mathbf{q} \|\mathbf{q}\|$: two nonzeros per cell, supplied to `TSSetRHSJacobianP`.

Verification. Three layers, all against central finite differences with pre-registered tolerances (Table 1): an edge-level harness that differences the flux routine itself on sampled states (with a Richardson self-check of the FD order); full-matrix comparisons of the assembled Jacobian against a matrix-coloring FD Jacobian of the actual RHS on three state/boundary combinations, boundary rows included; and end-to-end gradient checks through the adjoint (next section). A finite-difference coloring Jacobian (`numerics.jacobian: fd`) remains available permanently as the verification baseline.

4 Adjoint sensitivities

For gauge observations y_k at times t_k with observation operator H (one row per gauge, selecting water height) and error variance σ^2 ,

$$J(n) = \frac{1}{2\sigma^2} \sum_k \|Hu(t_k) - y_k\|^2. \quad (4)$$

`TSAdjoint` [13] differentiates the discrete time stepper, so gradients are exact to roundoff for the computed trajectory — the property that makes the FD gates in Table 1 meaningful. Multi-time observations are handled by a segmented backward sweep: the adjoint is advanced between observation times with `TSAdjointSetSteps` and receives a jump $H^T \sigma^{-2} r_k$ at each t_k (Algorithm 1 in Section 5). One sweep yields both dJ/du_0 (state sensitivity) and dJ/dn (per-cell parameter gradient, via the two-nonzero parameter Jacobian).

Two integrator facts constrain the design. PETSc’s plain forward-Euler integrator implements no adjoint, so adjoint-path configurations use the Runge–Kutta family. And for implicit integrators the discrete-adjoint gradient is only as exact as the inner solves: with default SNES/KSP tolerances the Manning gradient checked at 2×10^{-4} ; tightening the tolerances recovered 5×10^{-6} (Table 1).

5 Calibration twin experiments

The complete calibration loop is summarized in Algorithms 1–3. It is a strong-constraint 4D-Var iteration: the model is enforced exactly (the only controls are the Manning field and, optionally, the initial state), and each objective evaluation performs one checkpointed forward sweep that records the gauge residuals r_k at the observation times, followed by one backward sweep of the discrete adjoint (Algorithm 2). The backward sweep carries two vectors with distinct roles. The *adjoint state* λ holds $\partial J_{\text{mis}}/\partial u(t)$ at the current point of the backward traversal: it is seeded with the terminal observation sensitivity $\sigma^{-2} H^T r_K$, propagated one step at a time by the transposed Jacobian of the forward step map (the backward “model step”), and incremented by a jump $\sigma^{-2} H^T r_k$ each time the sweep crosses an observation time, because the state at that time enters the misfit directly. The *parameter sensitivity* μ starts at zero and accumulates, at every step, the chain-rule contribution $(\partial \Phi_s/\partial n)^T \lambda$ of that step’s Manning dependence; when the sweep reaches $t = 0$, μ is the complete misfit gradient dJ_{mis}/dn and λ is dJ_{mis}/du_0 . Both transposed Jacobians are the analytic operators of Sections 3–4: the assembled RHS Jacobian for the state propagation and the two-nonzero parameter Jacobian for the μ accumulation, applied within `TSAdjoint`’s discrete adjoint of the chosen integrator. The per-iteration cost is therefore one forward plus one adjoint solve, independent of the number of parameters. The outer update is TAO’s bound-constrained limited-memory quasi-Newton method BLMVM [14] (Algorithm 3), and the loop terminates on the standard bound-constrained test: the projected gradient g^P — the gradient with components pointing out of the feasible box zeroed — drops below an absolute tolerance, or an iteration budget is reached. The same loop serves every parameterization in this paper: per-region, per-cell, and land-cover-class fields differ only in the expansion of the parameter vector onto cells (line 3 of Algorithm 1) and the corresponding summation of μ back onto parameters.

All experiments use a wet planar dam-break configuration (50 cells, two regions; synthetic noise-free observations; TAO/BLMVM with bounds $n \in [0.01, 0.2]$).

Per-region. Truth $n = (0.03, 0.06)$ by region, initial guess 0.02 uniform, terminal-time observations at every second cell. BLMVM recovers both values to 4.1×10^{-7} maximum relative error.

Algorithm 1 Strong-constraint 4D-Var calibration of the Manning field, as implemented with TAO/BLMVM and TSAdjoint. The window map $M_{t_{k-1} \rightarrow t_k}$ is the composition of m steps of the discrete forward stepper (explicit RK, backward Euler, or ARK-IMEX); Algorithm 2 differentiates exactly this stepper, so the gradient is exact to roundoff (and inner-solver tolerance, Section 4) for the computed trajectory.

Require: prior/initial guess n_0 , bounds $[n_{\min}, n_{\max}]$, observations y_k at times $t_k = k m \Delta t$ ($k = 1, \dots, K$), gauge operator H , error σ , Tikhonov weight β , gradient tolerance ε_g , iteration budget $I_{\max}$

- 1: $n \leftarrow n_0$
- 2: **repeat**
- 3: set the model's cellwise drag coefficient from $n \triangleright$ identity for per-cell n ; constant over each region or land-cover class for the reduced parameterizations
- 4: $u \leftarrow u_0$ $\triangleright$ fixed initial state (strong constraint)
- 5: **for** $k = 1, \dots, K$ **do** $\triangleright$ forward sweep; every step checkpointed
- 6: $u \leftarrow M_{t_{k-1} \rightarrow t_k}(u; n)$
- 7: $r_k \leftarrow Hu - y_k$
- 8: **end for**
- 9: $J \leftarrow \frac{1}{2\sigma^2} \sum_{k=1}^K \|r_k\|^2 + \frac{\beta}{2} \|n - n_0\|^2$
- 10: $(\lambda, \mu) \leftarrow \text{ADJOINTSWEEP}(\{r_k\})$ $\triangleright$ Algorithm 2: $\mu = dJ_{\text{mis}}/dn$
- 11: $g \leftarrow \mu + \beta(n - n_0)$
- 12: $n \leftarrow \text{BLMVMSTEP}(n, J, g)$ $\triangleright$ Algorithm 3
- 13: **until** $\|g^P\| \leq \varepsilon_g$ or $I_{\max}$ iterations
- 14: **return** n

Algorithm 2 ADJOINTSWEEP: segmented discrete-adjoint backward sweep (TSAdjointSolve with TSAdjointSetSteps). Φ_s denotes the forward map of step s , $u_s = \Phi_s(u_{s-1}; n)$; its transposed Jacobians are built from the assembled RHS Jacobian $\partial f / \partial u$ (Section 3) and the parameter Jacobian $\partial f / \partial n$ according to the integrator's discrete-adjoint formula.

Require: checkpointed trajectory $\{u_s\}_{s=0}^{Km}$, residuals $\{r_k\}_{k=1}^K$ at steps $s = km$, gauge operator H , error σ

Ensure: $\lambda = dJ_{\text{mis}}/du_0$, $\mu = dJ_{\text{mis}}/dn$

- 1: $\lambda \leftarrow \sigma^{-2} H^T r_K$ $\triangleright \lambda$ holds $\partial J_{\text{mis}} / \partial u(t_s)$ throughout
- 2: $\mu \leftarrow 0$
- 3: **for** $s = Km, \dots, 1$ **do**
- 4: restore u_{s-1} (and stage values) from the checkpointed trajectory
- 5: $\mu \leftarrow \mu + (\partial \Phi_s / \partial n)^T \lambda$ $\triangleright$ accumulate this step's Manning dependence
- 6: $\lambda \leftarrow (\partial \Phi_s / \partial u_{s-1})^T \lambda$ $\triangleright$ backward model step (transposed step Jacobian)
- 7: **if** $s - 1 = km$ for some $1 \leq k < K$ **then**
- 8: $\lambda \leftarrow \lambda + \sigma^{-2} H^T r_k$ $\triangleright$ observation jump: u_{km} enters J_{mis} directly
- 9: **end if**
- 10: **end for**
- 11: **return** (λ, μ)

Algorithm 3 BLMVMSTEP: one iteration of TAO’s bound-constrained limited-memory variable-metric method [14]. $P_{[n_{\min}, n_{\max}]}$ is componentwise clipping to the bound box; the curvature pairs $\{(s_i, y_i)\}$ persist across calls. The projected gradient used in the convergence test of Algorithm 1 is $g_i^P = g_i$ for n_i strictly inside the box, $\min(g_i, 0)$ at the lower bound, and $\max(g_i, 0)$ at the upper bound.

Require: iterate n , objective J , gradient g , stored pairs $\{(s_i, y_i)\}$, bounds $[n_{\min}, n_{\max}]$

- 1: $d \leftarrow -B g^P$ $\triangleright B \approx (\nabla^2 J)^{-1}$ by the L-BFGS two-loop recursion over the stored pairs
- 2: find $\alpha > 0$ with a projected line search on $n(\alpha) = P_{[n_{\min}, n_{\max}]}(n + \alpha d)$ satisfying sufficient decrease of J $\triangleright$ each trial α costs one forward-plus-adjoint evaluation (lines 3–11 of Algorithm 1)
- 3: $n^+ \leftarrow n(\alpha)$; evaluate g^+ at n^+
- 4: $s \leftarrow n^+ - n$; $y \leftarrow g^+ - g$; store (s, y) if $s^T y > 0$, discarding the oldest pair beyond the memory limit
- 5: **return** n^+

Observations	β	TAO its	converged	rel. L^2 error
terminal only	10^{-4}	688	yes	39.5%
8 times	10^{-4}	200 (cap)	no	19.9%
20 times	10^{-5}	1864	yes	8.0%

Table 2: Per-cell twin recovery vs. observability (planar dam break, 50 gauges, 150 parameters, 40-step window).

Per-cell with Tikhonov regularization. Truth is a two-zone field split at the domain midline; parameters are per-cell (three per observation in the densest case), with $J += \frac{\beta}{2} \|n - n_0\|^2$ about a constant prior. Recovery is assessed over observable cells (wet and moving at the final time; dry cells are untouched by friction and correctly remain at the prior). The controlling factor is observability (Table 2): a single terminal snapshot leaves the problem badly underdetermined (39.5% error at full convergence); eight observation times already halve the error (19.9%, and still descending when stopped at a 200-iteration cap); and 20 observation times over the same window reach 8.0%, the remainder being genuine null space held by the prior. The exact analytic Jacobian reduced the cost of an adjoint sweep by roughly $6\times$ relative to the FD-coloring Jacobian, making the converged experiments interactive (~ 2 minutes on one laptop core).

6 Implicit stepping, enabled and required

The explicit drag term is stiff in thin water: linearizing gives a stability limit $\Delta t \lesssim 2h^{4/3}/(gn^2|v|)$, which for sheet flow at centimeter depths is seconds — far below the gravity-wave CFL limit of kilometer-scale meshes. Our Hurricane Harvey test configuration (Houston at 1 km, $\Delta t = 30$ s) confirms this empirically: the explicit source treatment produces NaN within the first objective evaluation, in quantitative agreement with the analysis.

The assembled exact Jacobian makes the remedy nearly free: backward Euler via PETSc’s theta method, with SNES driven by the same Jacobian, carries the Harvey window at $\Delta t = 30$ s, and the theta-method adjoint passes the same FD gates (Table 1).

We also implemented the sharper tool: an ARK-IMEX splitting with the Manning drag as the stiff implicit part (`temporal: arkimex`). The implicit stage system is block-diagonal — an independent 3×3 Newton per cell, no global solve — while fluxes and bed slope remain explicit under

the gravity-wave CFL limit. Its adjoint passes the same gates (2.0×10^{-8} for dJ/du_0 , 2.5×10^{-6} for dJ/dn) and it carries the Harvey window at $\Delta t = 30$ s at 6.7 s per forward-plus-adjoint sweep, with gradients matching backward Euler to three digits. The splitting also runs under RDycore’s libCEED operator backend — the explicit side uses the friction-free source Q-functions prepared in PR #359, and the two backends agree to machine precision (5×10^{-16} relative) after 100 ARK-IMEX steps; a device-resident implicit part is the natural follow-up. Three implementation facts others will hit: PETSc’s ARKIMEX adjoint dereferences *both* the implicit- and explicit-part parameter Jacobians, so a zero explicit-part `TSSetRHSJacobianP` must be registered even when the parameter appears only implicitly; the embedded step-size controller must be disabled for finite-difference gradient validation, since perturbed solves otherwise take different step sequences (an apparent $\sim 10^{-4}$ gradient error that vanishes with fixed steps); and PETSc’s in-memory checkpointing trajectory cannot tolerate a *segmented* forward solve (one `TSSolve` per observation window): `TSSolve` sets its converged reason before the final trajectory write of every call, which the memory trajectory interprets as the end of the forward run, corrupting the checkpoint schedule at every interior observation time. The forward sweep must be a single `TSSolve` with observations recorded by a monitor; the segmented backward sweep is unaffected.

Checkpointed adjoints for long windows. A naive stored trajectory for a full 36-hour Hurricane Harvey window at the 30 m mesh’s CFL-limited $\Delta t = 0.25$ s (518,400 steps, 2.93M cells) would occupy tens of petabytes. PETSc’s checkpointing trajectory (`-ts_trajectory_type memory` with the revolve optimal-schedule library) replaces storage with recomputation, and we validated it across the full hierarchy: on the dam-break twin it reproduces the disk-trajectory calibration and gradient gates bit-for-bit for explicit RK, backward Euler, and ARK-IMEX (the implicit integrators need the same tight inner tolerances as in Section 4, since checkpoint recomputation re-solves the stage systems); and on the 30 m mesh a forward-plus-adjoint sweep with only *three* states checkpointed in memory matches the disk-trajectory gradient to all printed digits while running slightly faster (recomputation is cheaper than trajectory I/O at this scale) and in less memory. For the full window, a few hundred checkpoints (~ 14 GB aggregate) bound the recompute overhead by a factor of about four, making the long-window adjoint memory-feasible; the remaining cost is the forward compute itself, addressed next.

7 Hurricane Harvey demonstration

The per-cell twin runs on the Houston 1 km Hurricane Harvey mesh (2,746 cells, 4200 s window at $\Delta t = 30$ s): truth is a two-zone field split at the domain midline, observed at 343 synthetic gauges over 14 observation times, calibrated with implicit stepping and Tikhonov regularization ($\beta = 10^{-5}$). Every cell is wet and moving in this configuration, so all 2,746 parameters are observable. This configuration is deliberately gauge-sparse — 343 gauges for 2,746 parameters, an 8:1 ratio — and it exhibits the classic *semiconvergence* of underdetermined inverse problems (Figure 1). At 300 BLMVM iterations (~ 22 minutes on one laptop core, ~ 4.4 s per forward-plus-adjoint) the recovered map shows the two-zone truth structure cleanly at 26.3% relative L^2 error. Continuing the same weakly regularized optimization ($\beta = 10^{-5}$) to 2000 iterations *degrades* recovery to 38.0%: the misfit keeps decreasing while the parameters drift along observation-null directions, ultimately pinning cells against both bound constraints (Figure 1, right). Early stopping was acting as the effective regularizer. The behavior disappears when the problem is made well posed: the densely observed dam-break twin of Table 2 converges monotonically to 8%, and a properly regularized Houston run ($\beta = 10^{-4}$, 686 gauges) reaches 19.4% at a 1000-iteration budget

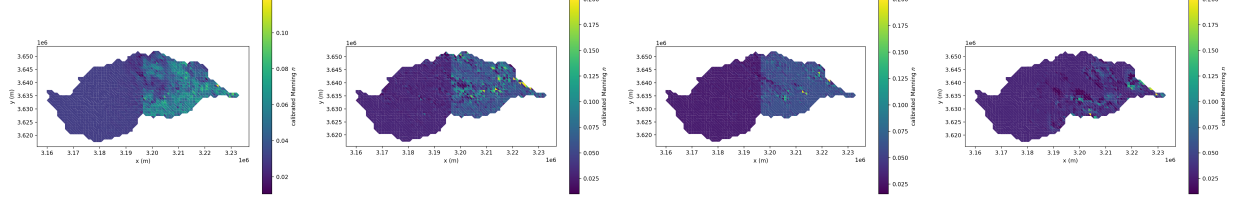


Figure 1: Regularization, semiconvergence, and network density on the Houston twin (2,746 parameters; truth: $n = 0.03$ west, $n = 0.06$ east of the midline; implicit stepping, 14 observation times). Far left: weak regularization ($\beta = 10^{-5}$, 343 gauges) stopped early at 300 iterations — 26.3% error. Center left: the same setup run to 2000 iterations — the misfit is lower but the field drifts along unobserved directions to 38.0% error and saturates the bounds $[0.01, 0.2]$. Center right: proper Tikhonov weight ($\beta = 10^{-4}$, 686 gauges), 1000 iterations — 19.4% error, no drift, sharp zone boundary. Far right: the *real* network — the 17 in-mesh USGS gauges with Harvey stage records, same $\beta = 10^{-4}$, 1000 iterations — the misfit falls 2×10^7 -fold while the field recovers to only 44.9%: no trace of the two-zone truth, near-prior almost everywhere with null-space structure pinned against both bounds.

with no drift (Figure 1, right) — the sharpest recovery of the three, with the zone boundary resolved at the truth midline. This is precisely the overfitting risk documented for distributed friction estimation [7], here reproduced in a controlled twin setting, and it motivates the regularization studies (total variation, land-cover priors, discrepancy-principle stopping) planned as follow-on work.

The real gauge network. The synthetic gauge sets above are a controlled ramp; the network that matters is the real one. Twenty USGS gauges fall inside the Houston mesh, seventeen with Hurricane Harvey stage records (HydroShare DOI 10.4211/hs.c037167e497546a1bc1508dfb32a9cff). Re-running the same twin with the observation operator built at those seventeen gauge cells — 238 observations ($17 \text{ gauges} \times 14 \text{ times}$) against 2,746 parameters — is a sobering experiment: BLMVM drives the misfit down by a factor of 2×10^7 , an essentially perfect fit of the observations, while the recovered field is 44.9% from the truth and saturates both bounds (Figure 1, far right). This is semiconvergence in its purest form: the real network’s null space is so large that data fit says almost nothing about parameter recovery, and no Tikhonov weight or stopping rule rescues a per-cell field from 238 observations. The practical conclusion for real-data calibration is to shrink the parameter space to match the network — the land-cover-class parameterization (~ 13 NLCD classes for this domain, overdetermined $18\times$ by the same observations) is the planned configuration, with the per-cell map reserved for densely observed settings. Calibration against the actual stage records (rather than twin-synthetic values at the real gauge locations) additionally requires the stage-datum QC completed after this draft was frozen, and is deliberately deferred.

8 Nondifferentiability catalog

Retrofitting an adjoint onto an operational flood code surfaces nonsmooth structure worth documenting:

- **Time step inside the RHS.** Operator-split friction treatments (here: semi-implicit and XQ2018) make f depend on Δt and on intermediate flux divergences; no consistent $\partial f / \partial u$

exists. The remedy is an explicit f plus a genuinely implicit integrator, not a smarter derivative.

- **Entropy-fix branches.** The critical-flow fix is piecewise smooth; branch-faithful one-sided derivatives pass FD checks even on transcritical states (Table 1).
- **Critical-outflow stagnation.** The outflow ghost map $h_g = (q^2/g)^{1/3}$ has an unbounded one-sided derivative at $q = 0$, and the inflow branch switches there: a genuine subgradient point, benign in practice because outflow boundaries carry outflow.
- **Wet/dry branches.** The RHS’s dry cutoffs are honored branch-faithfully in every block; fully wet validation cases keep FD comparisons clean, and dry cells are correctly insensitive to their roughness.

9 Conclusions and outlook

An exact, verified Jacobian turns out to be the single artifact from which everything else follows cheaply: adjoint state and parameter sensitivities, gradient-based calibration, and implicit stepping. The capability is production-ready in the sense that matters for an operational code — behind a flag, FD-gated in CI, and with the verification baseline (FD coloring) kept alive in the configuration system.

Next steps, in order: the Houston per-cell demonstration with real USGS Harvey gauges (datum QC pending); regularization studies (total variation, land-cover priors) for the ill-posedness that our observability results quantify; and coupling the gradient path to PyTorch through the existing PETSc–PyTorch bridge for learned-physics and hybrid data-assimilation experiments.

A closing note on cost. Checkpointing makes the long-window adjoint memory-feasible, but at the 30 m mesh’s CFL-limited $\Delta t = 0.25$ s a full 36-hour Hurricane Harvey gradient still costs several forward sweeps over half a million steps, and a many-iteration per-cell calibration at that resolution is not a defensible allocation. The affordable path stacks three reductions, all enabled by capabilities already in place: fully implicit stepping at CFL-free time steps (ARK-IMEX keeps the explicit flux CFL; backward Euler with the assembled Jacobian does not, trading a global Newton–Krylov solve per step for 20–100× fewer steps — making that solver scale is the next solver problem); land-cover-class parameterization (tens of TAO evaluations instead of hundreds); and calibration on the storm peak rather than the full window.

Open questions for internal review

[This section will be removed before submission. Please reply inline or by email — each question names who we think owns it.]

Software / engineering (Jeff)

1. **ARK-IMEX design intent.** PR #359 prepped the `SOURCE_ARK_IMEX` path (“bed friction moved to LHS”). We completed it as: Manning drag *alone* in the `IFunction` (per-cell block-diagonal `IJacobian`), bed slope and external sources explicit. Your CEED-side friction-free Q-functions are wired in and the two backends agree to machine precision. Does that match your intended split, do you have an ARKIMEX scheme/order preference (we use the PETSc default), and did you envision a device-resident (CEED Q-function) implicit part as the follow-up?

2. **Code placement and naming.** New files `swe_jacobian_petsc.c` and `swe_roe_flux_jacobian_petsc.h` (static-function header, mirroring `swe_roe_flux_petsc.h`); config keys `numerics.jacobian: none|fd|analytic` and `source.method: explicit|ark_imex`. Naming/placement objections before this hardens into a PR?
3. **Public API.** The adjoint driver currently reaches through `private/rdycoreimpl.h` (LETKF-driver pattern). Which calls deserve promotion to `rdycore.h` (`RDySetObservations`, `RDyAdjointSolve`, `RDyGetSensitivities?`), and are Fortran bindings needed at that point?
4. **Jacobian preallocation.** We use `DMCreateMatrix` under the DM’s closure-based adjacency, a superset of FV edge adjacency (extra stored zeros). Accept, or tighten the pattern?
5. **CI.** The 12 new tests need a PETSc with TSAdjoint (any recent version) and run in seconds. Any CI-matrix concerns (e.g., the CEED-only lanes correctly skip `numerics.jacobian` configs)?

Science (Gautam and team)

1. **Houston mesh CRS** (already asked): the projected coordinate system of `Houston1km_with_z.exo` — the one blocker for real-gauge calibration.
2. **Drag formulation on the adjoint path.** The new explicit treatment evaluates $\tau_b = gn^2 h^{-7/3} \mathbf{q} \|\mathbf{q}\|$ at the current state with the standard $h_{\min}$ cutoff and *plain* velocity reconstruction ($h_{\text{anuga}} = 0$ default). Physics sign-off? Should the ANUGA regularization be active for calibration runs?
3. **Observation choices for real-gauge work (v2).** Which of the 90 Houston-area USGS gauges do you trust for Harvey (backwater/tidal contamination?); stage vs. discharge as the misfit variable; and what observation error σ is realistic per gauge?
4. **Regularization prior.** Is a land-cover-derived $n_0(x)$ field available (or easy to produce) for the Houston mesh? The twin experiments use a constant prior.
5. **Truth-field realism.** Is the two-zone synthetic truth adequate for the paper’s twin experiments, or would you prefer a more realistic synthetic field (e.g., channel-vs-overbank)?
6. **Calibration windows and wet/dry.** Our Harvey window keeps the whole domain wet and moving. For longer or rain-driven windows, which events/phases would you calibrate on, given adjoint noise at drying fronts?
7. **Bounds.** Are $n \in [0.01, 0.2]$ physically sensible limits for Houston land cover?
8. **Venue and authorship** (already asked): target journal (WRR / JAMES / GMD) and the co-author list.

References

- [1] Yan Ding, Yafei Jia, and Sam S. Y. Wang. Identification of Manning’s roughness coefficients in shallow water flows. *Journal of Hydraulic Engineering*, 130(6):501–510, 2004.

- [2] Xianqiang Chen et al. Adjoint model of shallow water equations based on the finite volume method and unstructured meshes. *Abstract and Applied Analysis*, 2014:407468, 2014. Twin-experiment inversion of Manning roughness; velocity vs. stage observation sensitivity.
- [3] Asier Lacasta, Mario Morales-Hernández, Pilar García-Navarro, et al. A GPU accelerated adjoint-based optimizer for inverse modeling of the two-dimensional shallow water equations. *Computers & Fluids*, 2016.
- [4] Jérôme Monnier et al. DassFlow: Data assimilation for free-surface flows (DassHydro platform). <https://dasshydro.github.io>, 2025. 2D finite-volume shallow water with variational DA; Manning, bathymetry, and discharge inference incl. SWOT/AirSWOT and video-derived surface velocity.
- [5] Pierre Brisset, Jérôme Monnier, Pierre-André Garambois, and Hélène Roux. River discharge estimation from synthetic SWOT-type observations using variational data assimilation and the full Saint-Venant hydraulic model. *Journal of Hydrology*, 558:176–195, 2018.
- [6] Tuomas Kärnä et al. Efficient optimization of a regional water elevation model with an automatically generated adjoint. *Journal of Advances in Modeling Earth Systems*, 15, 2023. Spatially varying Manning friction calibrated for the Baltic Sea with Thetis/Firedrake/pyadjoint.
- [7] Simon C. Warder, Kevin J. Horsburgh, and Matthew D. Piggott. Adjoint-based sensitivity analysis for a numerical storm surge model. *Ocean Modelling*, 160:101766, 2021.
- [8] Zhi Li. Inunda: A GPU-native, agent-enabled, differentiable solver for high-resolution flood inundation modeling. arXiv:2607.09614, 2026. PyTorch local-inertial solver; roughness/bed/infiltration by gradient descent; Hurricane Harvey hindcast, Harris County TX.
- [9] Xiaofeng Liu et al. Scientific machine learning of flow resistance using universal shallow water equations with differentiable programming. *Water Resources Research*, 61, 2025. Hydrograd (Julia/SciML): AD-based Manning inversion and neural flow-resistance terms.
- [10] Elsa Cardoso-Bihlo and Alex Bihlo. A solver-in-the-loop framework for end-to-end differentiable coastal hydrodynamics. arXiv:2604.07129, 2026. AegirJAX: differentiable non-hydrostatic shallow-water solver in JAX.
- [11] Shengyu Kang, Jiabo Yin, and Dai Yamazaki. CaMa-Flood-GPU: A GPU-based hydrodynamic model implementation for scalable global simulations. *Geoscientific Model Development*, 19:5623–5640, 2026.
- [12] Tadd Bindas et al. Improving river routing using a differentiable Muskingum-Cunge model and physics-informed machine learning. *Water Resources Research*, 60, 2024. Differentiable routing inferring Manning roughness and channel geometry with neural networks.
- [13] Hong Zhang, Emil M. Constantinescu, and Barry F. Smith. PETSc TSAdjoint: A discrete adjoint ODE solver for first-order and second-order sensitivity analysis. *SIAM Journal on Scientific Computing*, 44(1):C1–C24, 2022.
- [14] Steven J. Benson and Jorge J. Moré. A limited memory variable metric method in subspaces and bound constrained optimization problems. Technical Report ANL/MCS-P909-0901, Argonne National Laboratory, Mathematics and Computer Science Division, 2001.